

```
import matplotlib.pyplot as plt
import math
```

```
def resolver_trelica_plana(nos, barras, cargas, apoios):
```

```
    N = len(nos)
```

```
    B = len(barras)
```

```
    # 1. Identificar Reações de Apoio
```

```
    reacoes = []
```

```
    for no_idx, tipo in apoios.items():
```

```
        if tipo == 'fixo':
```

```
            reacoes.append((no_idx, 'x'))
```

```
            reacoes.append((no_idx, 'y'))
```

```
        elif tipo == 'movel_y':
```

```
            reacoes.append((no_idx, 'y'))
```

```
        elif tipo == 'movel_x':
```

```
            reacoes.append((no_idx, 'x'))
```

```
    R = len(reacoes)
```

```
    num_incognitas = B + R
```

```
    num_equacoes = 2 * N
```

```
    # 2. Montagem da Matriz de Equilíbrio ( $Ax = b$ )
```

```
    A = [[0.0 for _ in range(num_incognitas)] for _ in range(num_equacoes)]
```

```
    b = [0.0] * num_equacoes
```

```
    for no_idx, (fx, fy) in cargas.items():
```

```
        b[2 * no_idx] = -fx
```

```
        b[2 * no_idx + 1] = -fy
```

```
    for i, (n1, n2) in enumerate(barras):
```

```
        dx = nos[n2][0] - nos[n1][0]
```

```
        dy = nos[n2][1] - nos[n1][1]
```

```
        L = math.sqrt(dx**2 + dy**2)
```

```
        cos, sen = dx/L, dy/L
```

```
        A[2*n1][i] = cos
```

```
        A[2*n1+1][i] = sen
```

```
        A[2*n2][i] = -cos
```

```
        A[2*n2+1][i] = -sen
```

```
    for j, (no_idx, eixo) in enumerate(reacoes):
```

```
        coluna = B + j
```

```
        if eixo == 'x':
```

```
            A[2*no_idx][coluna] = 1.0
```

```
        else:
```

```
            A[2*no_idx+1][coluna] = 1.0
```

3. Eliminação de Gauss com Pivoteamento Parcial

Criar matriz aumentada

```
for i in range(len(A)):
```

```
    A[i].append(b[i])
```

```
n_sis = len(A)
```

```
m_sis = num_incognitas
```

```
for i in range(min(n_sis, m_sis)):
```

```
    max_row = i
```

```
    for k in range(i + 1, n_sis):
```

```
        if abs(A[k][i]) > abs(A[max_row][i]):
```

```
            max_row = k
```

```
    A[i], A[max_row] = A[max_row], A[i]
```

```
    if abs(A[i][i]) > 1e-12:
```

```
        for k in range(i + 1, n_sis):
```

```
            fator = A[k][i] / A[i][i]
```

```
            for j in range(i, m_sis + 1):
```

```
                A[k][j] -= fator * A[i][j]
```

```
x_sol = [0.0] * m_sis
```

```
for i in range(min(n_sis, m_sis) - 1, -1, -1):
```

```
    if abs(A[i][i]) > 1e-12:
```

```
        soma = sum(A[i][j] * x_sol[j] for j in range(i + 1, m_sis))
```

```
        x_sol[i] = (A[i][m_sis] - soma) / A[i][i]
```

```
forcas_barras = x_sol[:B]
```

4. Saídas Gráficas

```
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
```

G1: Geometria Original

```
for i, (n1, n2) in enumerate(barras):
```

```
    axs[0,0].plot([nos[n1][0], nos[n2][0]], [nos[n1][1], nos[n2][1]], 'gray', marker='o')
```

```
    mx, my = (nos[n1][0]+nos[n2][0])/2, (nos[n1][1]+nos[n2][1])/2
```

```
    axs[0,0].text(mx, my, f'B{i}', color='red', fontsize=9)
```

```
axs[0,0].set_title("1. Geometria e Numeração")
```

G2: Mapa de Esforços (Calor)

```
for i, (n1, n2) in enumerate(barras):
```

```
    f = forcas_barras[i]
```

```
    cor = 'blue' if f > 0.01 else ('red' if f < -0.01 else 'green')
```

```
    axs[0,1].plot([nos[n1][0], nos[n2][0]], [nos[n1][1], nos[n2][1]], color=cor, lw=2)
```

```
axs[0,1].set_title("2. Esforços (Azul: Tração | Vermelho: Compr.)")
```

G3: Magnitude por Barra

```
axs[1,0].bar([f'B{i}' for i in range(B)], forcas_barras, color='skyblue')
```

```
axs[1,0].set_title("3. Magnitude das Forças (N)")
axs[1,0].tick_params(axis='x', rotation=45)
```

```
# G4: Histograma de Distribuição
axs[1,1].hist(forcas_barras, bins=10, color='orange', edgecolor='black')
axs[1,1].set_title("4. Distribuição das Forças")
```

```
plt.tight_layout()
plt.show()
```

```
return forcas_barras
```

```
# --- TESTE PADRÃO (TRELIÇA WARREN 4 NÓS) ---
```

```
nos_teste = [(0,0), (2,0), (4,0), (2,2)]
barras_teste = [(0,1), (1,2), (0,3), (1,3), (2,3)]
cargas_teste = {1: (0, -1000)} # 1000N para baixo no nó central
apoios_teste = {0: 'fixo', 2: 'movel_y'}
```

```
# Execução
```

```
resultados = resolver_trelica_plana(nos_teste, barras_teste, cargas_teste, apoios_teste)
```

```
# Exibição dos resultados numéricos
```

```
print(f"{'Barra':<10} | {'Força (N)':<12} | {'Estado'}")
```

```
print("-" * 35)
```

```
for i, f in enumerate(resultados):
```

```
    estado = "Tração" if f > 0.01 else ("Compressão" if f < -0.01 else "Nula")
```

```
    print(f"B{i:<9} | {f:<12.2f} | {estado}")
```